



VNiVERSIDAD
D SALAMANCA

Elaboración de libros electrónicos mediante código y asistentes de Inteligencia Artificial

Curso para las Facultades de Ciencias y de Ciencias Químicas

Álvaro Lozano Murciego y André Filipe Sales Mendes

Universidad de Salamanca

ÍNDICE DE CONTENIDOS

I	Sección 1 - Fundamentos	3
1	Sección 1: Fundamentos	5
1.1	¿Por Qué Aprender Aprendizaje Automático?	5
1.2	¿Qué Encontrarás en Esta Sección?	5
2	1.1 Historia y Evolución del Aprendizaje Automático	7
2.1	Introducción	7
2.2	Los Orígenes (1950s)	7
2.3	El Conexionismo Temprano (1950s-1960s)	7
2.4	El Invierno de la IA (1970s-1980s)	8
2.5	El Resurgimiento (1990s-2000s)	8
2.6	Deep Learning (2010s)	8
2.7	La Era de los Transformers y LLMs (2020+)	9
2.8	Lecciones Clave de la Historia	9
2.9	Resumen	10
3	1.2 Conceptos Fundamentales	11
3.1	Introducción	11
3.2	Datos, Características y Etiquetas	11
3.3	Overfitting y Underfitting	12
3.4	Validación Cruzada	13
3.5	Métricas de Evaluación	14
3.6	El Pipeline de Aprendizaje Automático	15
3.7	Resumen Conceptual	17
3.8	Actividad Práctica	18
II	Sección 2 - Aprendizaje Supervisado	19
4	Sección 2: Aprendizaje Supervisado	21
4.1	¿Qué es el Aprendizaje Supervisado?	21
4.2	Dos Tipos de Problemas	21
4.3	¿Qué Encontrarás en Esta Sección?	22
4.4	Estructura Recomendada	22
4.5	Objetivos de Aprendizaje	22
5	2.1 Regresión	25
5.1	Introducción	25
5.2	Regresión Lineal: Lo Básico	25
5.3	Mínimos Cuadrados Ordinarios (OLS)	26

5.4	Descenso de Gradiente	27
5.5	Regresión Polinomial	29
5.6	Regularización: Ridge y Lasso	30
5.7	Resumen	31
5.8	Próximo	31
6	2.2 Clasificación	33
6.1	Introducción	33
6.2	Regresión Logística	33
6.3	Matriz de Confusión	34
6.4	Métricas de Evaluación	35
6.5	Árboles de Decisión	36
6.6	Random Forest	37
6.7	Support Vector Machines (SVM)	38
6.8	Gradient Boosting (XGBoost, LightGBM)	38
6.9	Redes Neuronales: Introducción	39
6.10	Comparación de Algoritmos	41
6.11	Pauta para Elegir Algoritmo	41
6.12	Resumen	41
III	Sección 3 - Aprendizaje No Supervisado	43
7	Sección 3: Aprendizaje No Supervisado	45
7.1	¿Qué es Aprendizaje No Supervisado?	45
7.2	Tres Tipos Principales	45
7.3	¿Qué Encontrarás en Esta Sección?	46
7.4	Desafío Principal: Evaluación	46
7.5	Aplicaciones Reales	47
7.6	Estructura Recomendada	47
7.7	Objetivos de Aprendizaje	47
7.8	Nota Importante	48
8	3.1 Clustering	49
8.1	Introducción	49
8.2	K-Means	49
8.3	Eligiendo K: El Método del Codo	51
8.4	Clustering Jerárquico	52
8.5	DBSCAN (Density-Based Spatial Clustering)	53
8.6	Métrica de Evaluación: Silhueta	55
8.7	Resumen	55
9	3.2 Reducción de Dimensionalidad	57
9.1	Introducción	57
9.2	PCA: Principal Component Analysis	57
9.3	t-SNE (t-Distributed Stochastic Neighbor Embedding)	59
9.4	UMAP (Uniform Manifold Approximation and Projection)	61
9.5	Aplicaciones de Reducción de Dimensionalidad	61
9.6	Maldición de la Dimensionalidad	62
9.7	Resumen	62
10	3.3 Detección de Anomalías	63
10.1	Introducción	63
10.2	Conceptos Fundamentales	63
10.3	Métodos de Detección	64

10.4 Evaluación de Anomalías 66

10.5 Aplicaciones Reales 67

10.6 Implementación Conceptual: Isolation Forest 68

10.7 Comparación de Métodos 69

10.8 Resumen 69

10.9 Reflexión Final 69

Bienvenido al curso **Fundamentos e Introducción al Aprendizaje Automático** diseñado para estudiantes de las Facultades de Ciencias y Ciencias Químicas de la Universidad de Salamanca.

Propósito del Libro

Este libro te guía en un viaje desde los conceptos estadísticos clásicos hasta los modelos modernos de aprendizaje profundo. El objetivo es que **comprendas cómo las máquinas aprenden de los datos** y que seas capaz de construir, entrenar y evaluar modelos que resuelvan problemas reales.

Estructura del Contenido

El libro se divide en tres secciones principales:

Sección 1: Fundamentos

Comprender el cambio de paradigma: de la programación con reglas fijas al aprendizaje basado en datos.

- Origen histórico de la IA y el aprendizaje automático
- Conceptos fundamentales: datos, características, etiquetas
- Métrica de evaluación y validación cruzada

Sección 2: Aprendizaje Supervisado

Aprende a modelar variables continuas (regresión) y categóricas (clasificación).

- Regresión lineal y polinomial
- Clasificadores lineales y no lineales
- Redes neuronales

Sección 3: Aprendizaje No Supervisado

Descubre patrones ocultos en datos sin etiquetas.

- Clustering (agrupamiento)
- Reducción de dimensionalidad
- Detección de anomalías

¿A Quién va Dirigido?

Este curso está diseñado para:

- Estudiantes de grado en Ciencias, Física, Estadística, Informática o Ingenierías
- Cualquier persona con interés en el aprendizaje automático y datos
- Profesionales que desean actualizar sus habilidades

Requisitos Previos

Aunque el libro es autónomo, es útil tener:

- Conocimientos básicos de Python o disposición a aprender
- Familiaridad con álgebra lineal y cálculo (pero no es imprescindible)
- Pensamiento analítico y paciencia con los conceptos matemáticos

Cómo Usar Este Libro

Cada capítulo incluye:

- **Conceptos teóricos** explicados de forma clara
- **Ejemplos prácticos** con código Python ejecutable
- **Visualizaciones** para entender mejor los resultados
- **Ejercicios** para reforzar lo aprendido

Se recomienda seguir el orden establecido, aunque puedes consultar secciones específicas según tus necesidades.

¡Comenzamos! Dirígete a la Sección 1: Fundamentos para empezar tu viaje en el aprendizaje automático.

Part I

Sección 1 - Fundamentos

SECCIÓN 1: FUNDAMENTOS

Bienvenido a la primera sección del curso. Aquí comprenderemos los principios básicos del aprendizaje automático y cómo difiere del enfoque tradicional de programación.

1.1 ¿Por Qué Aprender Aprendizaje Automático?

En la programación tradicional, un programador escribe explícitamente todas las reglas que un programa debe seguir:

```
si temperatura > 30:  
    encender_aire_acondicionado()  
si temperatura < 15:  
    encender calefaccion()
```

Pero muchos problemas del mundo real son demasiado complejos para describirlos con reglas fijas. Por ejemplo:

- ¿Cómo distinguir automáticamente un gato de un perro en una fotografía?
- ¿Cómo predecir si una transacción bancaria es fraudulenta?
- ¿Cómo clasificar correos electrónicos como spam o no spam?

El aprendizaje automático invierte el paradigma: en lugar de escribir las reglas, le enseñamos al programa a *aprender* las reglas a partir de ejemplos.

1.2 ¿Qué Encontrarás en Esta Sección?

1. **Historia y Evolución:** Conoceremos cómo nació la IA y cómo evolucionó hasta llegar a los modelos modernos que usamos hoy.
2. **Conceptos Fundamentales:** Aprenderemos el vocabulario técnico esencial para trabajar con modelos de aprendizaje automático.

Estos fundamentos son críticos: sin una comprensión sólida de estos conceptos, será difícil entender las técnicas más avanzadas de las secciones posteriores.

Empecemos: dirígete al Capítulo 1.1 para explorar la historia fascinante del aprendizaje automático.

1.1 HISTORIA Y EVOLUCIÓN DEL APRENDIZAJE AUTOMÁTICO

2.1 Introducción

Entender la historia del aprendizaje automático nos ayuda a comprender por qué ciertos enfoques funcionan mejor que otros y por qué la investigación actual se enfoca en ciertas direcciones.

2.2 Los Orígenes (1950s)

2.2.1 La Pregunta Fundamental

En 1950, Alan Turing propuso una pregunta revolucionaria en su artículo *“Computing Machinery and Intelligence”*: **¿Pueden pensar las máquinas?**

Aunque el artículo no trataba específicamente sobre aprendizaje automático, planteó la cuestión fundamental: si una máquina pudiera comportarse de forma indistinguible de un humano, ¿habría alguna razón para decir que no piensa?

2.2.2 Las Neuronas Computacionales

En 1943, Warren McCulloch y Walter Pitts publicaron el primer modelo matemático de una neurona biológica. Su trabajo mostró que redes de estas neuronas simples podrían, en teoría, computar cualquier función lógica.

Este descubrimiento fue fundamental: demostró que **máquinas basadas en componentes simples podrían ser capaces de aprender**.

2.3 El Conexionismo Temprano (1950s-1960s)

2.3.1 El Perceptrón

En 1958, Frank Rosenblatt desarrolló el **Perceptrón**, el primer algoritmo de aprendizaje automático práctico. Era una máquina capaz de:

- Aprender a clasificar patrones simples
- Mejorar su desempeño con más ejemplos
- Generalizar a ejemplos nuevos

El optimismo era enorme. Los periódicos hablaban de máquinas que podrían pensar, ver y hasta reproducirse. Parecía que la inteligencia artificial estaba al alcance.

2.4 El Invierno de la IA (1970s-1980s)

2.4.1 El Fracaso de las Expectativas

Sin embargo, el Perceptrón tenía una limitación crucial: **no podía aprender patrones no lineales**, incluso patrones simples como la operación XOR (o exclusivo).

En 1969, Marvin Minsky y Seymour Papert publicaron un análisis matemático demostrando estas limitaciones. El desinterés resultante llevó a un período conocido como el “**Invierno de la IA**”.

2.4.2 La Era de los Sistemas Expertos

Durante este período, la IA simbólica dominó. Los investigadores intentaban capturar el conocimiento en sistemas de reglas “si-entonces”:

```
si sintomas == {fiebre, tos, dolor_garganta}:  
    diagnóstico = "gripe"
```

Este enfoque funcionaba para dominios específicos pero fallaba al escalar a problemas complejos.

2.5 El Resurgimiento (1990s-2000s)

2.5.1 Cambio de Paradigma

Lentamente, los investigadores regresaron al aprendizaje automático. Las razones fueron:

- **Más datos disponibles:** internet había generado enormes cantidades de datos
- **Computadoras más potentes:** ahora era posible entrenar modelos complejos
- **Nuevas técnicas:** algoritmos como Support Vector Machines (SVM), Random Forests y Gradient Boosting

Se descubrió que el aprendizaje estadístico, en lugar de la IA simbólica, era el camino correcto.

2.5.2 Validación cruzada y evaluación rigurosa

Durante este período también se desarrollaron métodos rigurosos para evaluar modelos, incluyendo validación cruzada, que veremos en detalle en el próximo capítulo.

2.6 Deep Learning (2010s)

2.6.1 La Revolución de las Redes Profundas

A principios de los 2010s, ocurrió algo sorprendente: agregando múltiples capas de neuronas (“profundidad”) y usando técnicas de optimización modernas, fue posible entrenar redes neuronales complejas.

En 2012, una red neuronal profunda (AlexNet) ganó la competencia ImageNet de clasificación de imágenes, superando significativamente a todos los métodos anteriores. Fue el momento en que el mundo reconoció que el aprendizaje profundo funcionaba.

2.6.2 Por Qué Funcionan las Redes Profundas

Las redes profundas aprenden **representaciones en capas sucesivas**:

- Las primeras capas aprenden características simples (bordes, texturas)
- Las capas intermedias combinan estas en características más complejas
- Las últimas capas aprenden conceptos abstractos (ojos, orejas, patrones)

Esta estructura jerárquica refleja cómo el cerebro humano procesa información.

2.7 La Era de los Transformers y LLMs (2020+)

2.7.1 Transformers

En 2017, se introdujo la arquitectura **Transformer**, que revolucionó el procesamiento de lenguaje natural. A diferencia de las redes recurrentes anteriores, los Transformers podían procesar secuencias en paralelo, haciéndolos más eficientes y escalables.

2.7.2 Large Language Models (LLMs)

Combinando Transformers con enormes cantidades de datos de internet y poder computacional, se desarrollaron modelos como GPT, que pueden:

- Generar texto coherente
- Responder preguntas
- Realizar múltiples tareas sin reentrenamiento específico

Estos modelos representan el estado del arte actual en aprendizaje automático.

2.8 Lecciones Clave de la Historia

1. **El progreso no es lineal**: ha habido períodos de entusiasmo excesivo seguidos de desencanto
2. **Los datos y la potencia computacional importan**: muchos algoritmos de hoy existían en forma teórica hace décadas, pero solo funcionan bien con suficientes recursos
3. **El cambio de paradigma**: hemos pasado de reglas explícitas → estadística clásica → redes profundas
4. **La evolución continúa**: las técnicas que aprendas hoy probablemente evolucionarán en los próximos años

2.9 Resumen

- El aprendizaje automático nació como respuesta a la pregunta: ¿pueden las máquinas aprender?
 - Ha pasado por ciclos de hype y desencanto
 - Los avances recientes se deben a más datos, mejor hardware y algoritmos innovadores
 - El aprendizaje profundo y los LLMs son el estado del arte actual
-

Siguiente: En el Capítulo 1.2, aprenderemos los conceptos fundamentales que necesitas para trabajar con cualquier modelo de aprendizaje automático.

1.2 CONCEPTOS FUNDAMENTALES

3.1 Introducción

Para trabajar efectivamente con modelos de aprendizaje automático, necesitamos dominar cierta terminología y comprensión de conceptos clave. Este capítulo construye el vocabulario que usaremos a lo largo del curso.

3.2 Datos, Características y Etiquetas

3.2.1 Estructura Básica

En aprendizaje automático, trabajamos con **conjuntos de datos** que tienen una estructura específica:

Muestra	Feature	Feature	Feature	Label
	1	2	3	
Ejemplo	Edad	Peso	Altura	Tipo
1	25	70	175	Gato
2	8	4	90	Gato
3	35	80	180	Perro

3.2.2 Definiciones

- **Muestra (Sample):** Un punto de datos individual. En el ejemplo anterior, cada fila es una muestra.
- **Características (Features):** Los atributos o variables que describen una muestra. También llamadas “variables independientes”.
- **Etiqueta (Label):** El objetivo que queremos predecir. También llamada “variable dependiente” o “respuesta”.

3.2.3 Ejemplo: Predicción de Precios de Vivienda

```
# Características: pueden ser números, categorías, etc.
features = [
    "metros_cuadrados",
    "numero_habitaciones",
    "ubicacion",
    "año_construccion"
]

# Etiqueta: lo que queremos predecir
label = "precio"
```

3.3 Overfitting y Underfitting

3.3.1 El Trade-off Fundamental

El error fundamental en aprendizaje automático es el **trade-off sesgo-varianza**. Está relacionado con dos problemas:

Underfitting (Subajuste)

Cuando el modelo es demasiado simple:

- El modelo no aprende los patrones reales de los datos
- Tiene **alto sesgo**: siempre comete el mismo tipo de error
- Funciona mal tanto en entrenamiento como en prueba

Overfitting (Sobreajuste)

Cuando el modelo es demasiado complejo:

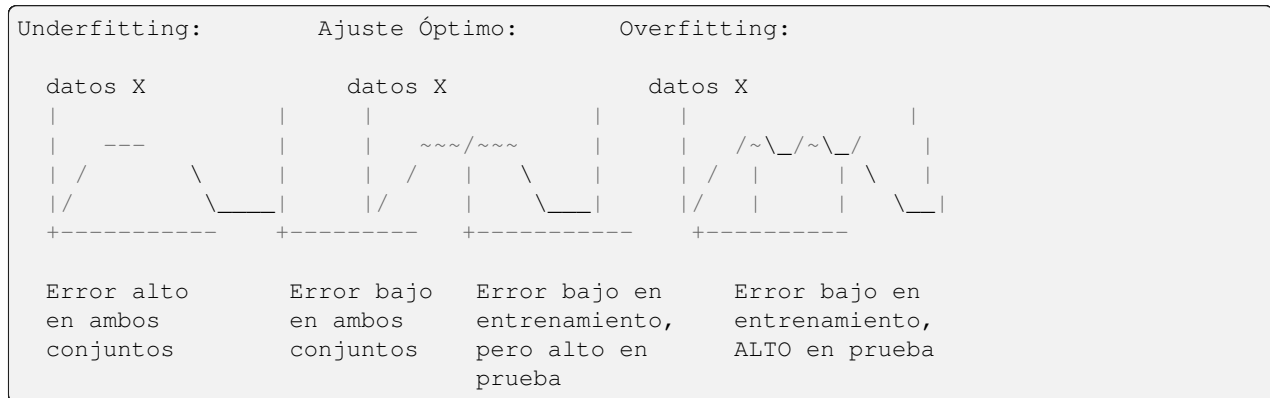
- El modelo memoriza los datos de entrenamiento, incluyendo el ruido
- Tiene **alta varianza**: los pequeños cambios en datos causan grandes cambios en predicciones
- Funciona bien en entrenamiento pero mal en datos nuevos

Note

La meta: Encontrar el punto óptimo donde el modelo generaliza bien a datos nuevos.

3.3.2 Visualización del Concepto

Imagina que queremos ajustar datos con una curva:



3.4 Validación Cruzada

3.4.1 El Problema

Cuando entrenamos un modelo, necesitamos evaluar si generaliza bien. Pero si evaluamos en los mismos datos donde entrenamos, no sabemos realmente qué tal le irá con datos nuevos.

3.4.2 La Solución: K-Fold Cross-Validation

En lugar de usar un único conjunto de prueba, dividimos los datos en K partes (folds):

1. Entrenar en K-1 folds
2. Evaluar en el fold restante
3. Repetir K veces, usando cada fold como conjunto de prueba

Ventajas:

- Usa todos los datos para entrenamiento y prueba
- Da una estimación más confiable del desempeño real
- Reduce la varianza de nuestra estimación

3.4.3 Ejemplo: 5-Fold Cross-Validation

```
Fold 1: [TRAIN | TRAIN | TRAIN | TRAIN | TEST]
Fold 2: [TRAIN | TRAIN | TRAIN | TEST | TRAIN]
Fold 3: [TRAIN | TRAIN | TEST | TRAIN | TRAIN]
Fold 4: [TRAIN | TEST | TRAIN | TRAIN | TRAIN]
Fold 5: [TEST | TRAIN | TRAIN | TRAIN | TRAIN]
```

Calculamos el error en cada fold y promediamos.

3.5 Métricas de Evaluación

3.5.1 Para Clasificación

Cuando predecimos una categoría (gato vs perro), usamos diferentes métricas:

Matriz de Confusión

		Predicción	
		Positivo	Negativo
Actuale	P	TP	FN
	N	FP	TN

- **TP (Verdadero Positivo):** Correctamente identificado como positivo
- **FP (Falso Positivo):** Incorrectamente identificado como positivo
- **TN (Verdadero Negativo):** Correctamente identificado como negativo
- **FN (Falso Negativo):** Incorrectamente identificado como negativo

Precisión (Precision)

$$\text{Precisión} = \frac{TP}{TP + FP}$$

Pregunta: De todos los casos que predijimos como positivos, ¿cuántos realmente lo eran?

Recall (Sensibilidad)

$$\text{Recall} = \frac{TP}{TP + FN}$$

Pregunta: De todos los casos que realmente eran positivos, ¿cuántos encontramos?

F1-Score

$$F1 = 2 \times \frac{\text{Precisión} \times \text{Recall}}{\text{Precisión} + \text{Recall}}$$

Combina precisión y recall en una única métrica.

AUC-ROC

Mide la capacidad del modelo de distinguir entre clases a diferentes umbrales de decisión. Un AUC de 1.0 es perfecto, 0.5 es aleatorio.

3.5.2 Para Regresión

Cuando predecimos un valor continuo (precio, temperatura), usamos:

Error Cuadrático Medio (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Donde:

- y_i es el valor real
- $\hat{y}_i$ es el valor predicho
- n es el número de muestras

Desventaja: Las unidades están al cuadrado.

Error Absoluto Medio (MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Ventaja: Las unidades son iguales a la variable original.

Raíz del Error Cuadrático Medio (RMSE)

$$RMSE = \sqrt{MSE}$$

Es MSE pero en unidades originales.

3.6 El Pipeline de Aprendizaje Automático

Crear un modelo de aprendizaje automático no es solo ejecutar un algoritmo. Es un proceso:

```

1. RECOLECCIÓN DE DATOS
  ↓
2. EXPLORACIÓN Y LIMPIEZA
  ↓
3. INGENIERÍA DE CARACTERÍSTICAS
  ↓
4. DIVISIÓN TRAIN/TEST
  ↓
5. SELECCIÓN DEL MODELO
  ↓
6. ENTRENAMIENTO
  ↓
7. EVALUACIÓN
  ↓
8. AJUSTE DE HIPERPARÁMETROS
  ↓
9. EVALUACIÓN FINAL
  
```

(continues on next page)

↓
10. DESPLIEGUE

Vamos a explicar brevemente cada paso:

3.6.1 1. Recolección de Datos

- Obtener datos relevantes para el problema
- Puede venir de bases de datos, APIs, sensores, etc.

3.6.2 2. Exploración y Limpieza

- Entender la estructura de los datos
- Identificar y manejar valores faltantes (missing values)
- Detectar y tratar outliers (valores atípicos)
- Verificar distribuciones

3.6.3 3. Ingeniería de Características

- Crear nuevas características útiles
- Escalar características numéricas
- Codificar categorías
- Seleccionar características relevantes

3.6.4 4. División Train/Test

- Separar datos en conjunto de entrenamiento y prueba
- Típicamente 70-80% entrenamiento, 20-30% prueba
- Usar validación cruzada para mayor confiabilidad

3.6.5 5. Selección del Modelo

- Elegir qué algoritmo usar
- Considerar el tipo de problema (regresión, clasificación, etc.)
- Considerar complejidad vs interpretabilidad

3.6.6 6. Entrenamiento

- Ejecutar el algoritmo en los datos de entrenamiento
- El modelo aprende los patrones

3.6.7 7. Evaluación

- Evaluar el desempeño en datos de prueba
- Usar métricas apropiadas

3.6.8 8. Ajuste de Hiperparámetros

- Si el desempeño no es satisfactorio, ajustar parámetros del modelo
- Volver al paso 6 y reentrenar

3.6.9 9. Evaluación Final

- Evaluar el mejor modelo en un conjunto de datos completamente nuevo

3.6.10 10. Despliegue

- Poner el modelo en producción
- Monitorear su desempeño en el mundo real

3.7 Resumen Conceptual

Concepto	Definición
Muestra	Un punto individual en el conjunto de datos
Características	Variables de entrada que describen una muestra
Etiqueta	Variable de salida que queremos predecir
Overfitting	Memorizar datos de entrenamiento, mal en prueba
Underfitting	Modelo demasiado simple, mal en ambos
Validación Cruzada	Técnica para evaluar desempeño sin sesgo
Precisión	Ratio de positivos predichos correctamente
Recall	Ratio de positivos reales encontrados

3.8 Actividad Práctica

Piensa en un problema que te gustaría resolver con aprendizaje automático:

1. ¿Cuáles serían tus características?
2. ¿Cuál sería tu etiqueta?
3. ¿Es un problema de clasificación o regresión?
4. ¿Cómo dividirías los datos?

Siguiente: Ahora que entiendes los fundamentos, pasaremos a la Sección 2: Aprendizaje Supervisado, donde aprenderás a construir modelos prácticos.

Part II

Sección 2 - Aprendizaje Supervisado

SECCIÓN 2: APRENDIZAJE SUPERVISADO

Bienvenido a la segunda sección del curso. Aquí aprenderemos a construir modelos que aprenden a partir de ejemplos etiquetados.

4.1 ¿Qué es el Aprendizaje Supervisado?

En aprendizaje supervisado, tenemos:

- **Datos:** características de entrada (X)
- **Etiquetas:** respuestas correctas (y)
- **Objetivo:** aprender la relación $X \rightarrow y$

Es como aprender con un “supervisor” que te dice si tus respuestas son correctas.

4.2 Dos Tipos de Problemas

4.2.1 1. Regresión

Predecimos un **valor continuo** (un número real):

- Predecir el precio de una casa
- Predecir la temperatura de mañana
- Predecir la cantidad de lluvia
- Predecir el crecimiento de una población

4.2.2 2. Clasificación

Predecimos una **categoría o clase**:

- Diagnosticar si una persona tiene una enfermedad (sí/no)
- Clasificar emails como spam o no spam
- Reconocer si una imagen es un gato, perro o pájaro
- Predecir si un cliente va a abandonarla

4.3 ¿Qué Encontrarás en Esta Sección?

4.3.1 Capítulo 2.1: Regresión

- Regresión Lineal: el modelo más simple y fundamental
- Mínimos Cuadrados Ordinarios: cómo entrena una regresión lineal
- Gradiente Descendente: el algoritmo de optimización más importante
- Regresión Polinomial y Regularización: cómo hacer modelos más complejos

4.3.2 Capítulo 2.2: Clasificación

- Regresión Logística: clasificación binaria
- Matriz de Confusión y Métricas: cómo evaluar clasificadores
- Árboles de Decisión y Random Forests: modelos interpretables
- Support Vector Machines: separación óptima de clases
- Métodos Ensemble: combinar múltiples modelos
- Redes Neuronales: introducción a las redes profundas

4.4 Estructura Recomendada

Se recomienda aprender en este orden:

1. **Regresión Lineal** → entiende los fundamentos
2. **Gradiente Descendente** → entiende cómo los modelos aprenden
3. **Regresión Polinomial** → entiende cómo manejar no linealidad
4. **Regresión Logística** → extiende regresión a clasificación
5. **Algoritmos de Clasificación** → aprende múltiples enfoques
6. **Ensemble Methods** → combina la fortaleza de múltiples modelos
7. **Redes Neuronales** → el futuro del aprendizaje automático

4.5 Objetivos de Aprendizaje

Después de completar esta sección, podrás:

- ✓ Implementar regresión lineal desde cero
- ✓ Entender cómo funciona el gradiente descendente
- ✓ Aplicar regularización para evitar overfitting
- ✓ Elegir el algoritmo correcto para tu problema
- ✓ Evaluar modelos de clasificación correctamente
- ✓ Construir una red neuronal simple

Empecemos: Dirígete al Capítulo 2.1 para aprender sobre Regresión, la base de muchos modelos de aprendizaje automático.

2.1 REGRESIÓN

5.1 Introducción

La regresión es el problema de predecir un valor continuo. Empezamos con regresión lineal, el modelo más simple y fundamental del aprendizaje automático.

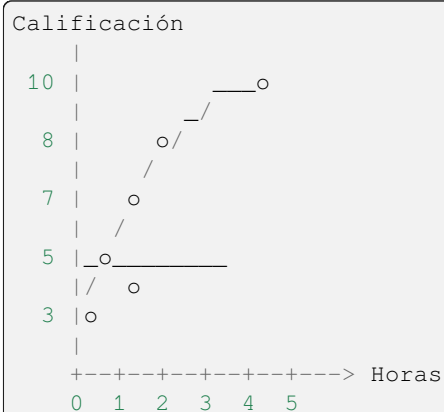
5.2 Regresión Lineal: Lo Básico

5.2.1 Intuición

Imagina que tienes datos sobre horas estudiadas y calificaciones de examen:

Horas Estudiadas	Calificación
1	3
2	5
3	7
4	8
5	10

La regresión lineal intenta encontrar una línea recta que se ajuste mejor a estos puntos:



La ecuación de la línea es: $\hat{y} = \beta_0 + \beta_1 x$

Donde:

- $\hat{y}$ es la predicción

- β_0 es el intercepto (dónde cruza el eje y)
- β_1 es la pendiente (qué tan rápido sube)
- x es la entrada

5.2.2 Generalización a Múltiples Características

Si tenemos múltiples características:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Por ejemplo, prediciendo precio de casa: $\text{\$precio} = \beta_0 + \beta_1 \times \text{metros}^2 + \beta_2 \times \text{habitaciones}$

5.3 Mínimos Cuadrados Ordinarios (OLS)

5.3.1 ¿Cómo encontramos los mejores β_0 y β_1 ?

La idea es minimizar la suma de los cuadrados de los **residuos** (errores):

```
Dato real (y)
|
o (dato)
| |
| ±-- residuo = y - ŷ
| |
----ŷ (predicción)
```

Objetivo: $\min \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Esta es la función de pérdida que queremos minimizar.

5.3.2 Solución Analítica

Para regresión lineal, existe una solución matemática exacta. En forma vectorial:

$$\beta = (X^T X)^{-1} X^T y$$

Donde:

- X es la matriz de características
- y es el vector de etiquetas
- β es el vector de coeficientes

5.3.3 Implementación Conceptual

```
import numpy as np

# Datos
X = np.array([[1], [2], [3], [4], [5]]) # Horas
y = np.array([3, 5, 7, 8, 10])          # Calificaciones

# Agregar columna de 1s para el intercepto
X = np.column_stack([np.ones(len(X)), X])

# Calcular beta = (X^T X)^-1 X^T y
beta = np.linalg.inv(X.T @ X) @ X.T @ y

print(f"Intercepto: {beta[0]}") # beta_0
print(f"Pendiente: {beta[1]}") # beta_1

# Hacer predicción para 6 horas
X_new = np.array([1, 6])
prediccion = X_new @ beta
print(f"Predicción para 6 horas: {prediccion}")
```

5.4 Descenso de Gradiente

5.4.1 Motivación

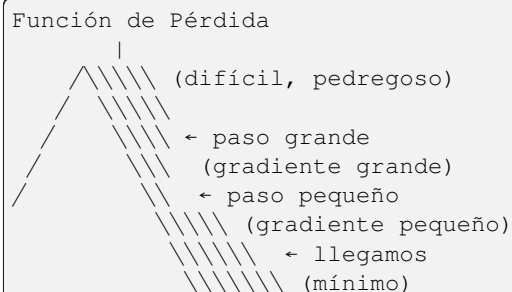
La solución OLS funciona bien para regresión lineal, pero tiene un problema: requiere invertir una matriz, lo que es computacionalmente costoso cuando tenemos millones de características.

Existe un método más general: **Descenso de Gradiente**, que funciona para cualquier tipo de modelo.

5.4.2 Intuición

Imagina que estás en una montaña nublada:

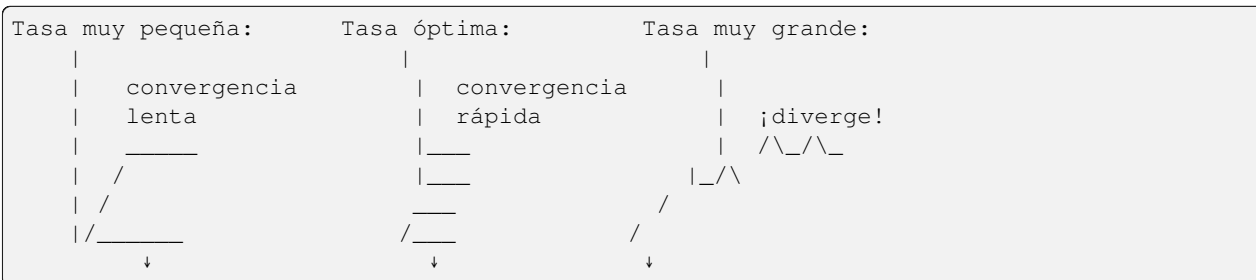
- No puedes ver el valle (el mínimo)
- Pero puedes sentir la pendiente bajo tus pies
- Si siempre das un paso en la dirección de la pendiente más pronunciada hacia abajo, eventualmente llegarás al valle



5.4.3 El Algoritmo

1. Inicializar β aleatoriamente
2. Repetir hasta convergencia:
 - Calcular el gradiente: $\nabla J = \frac{\partial J}{\partial \beta}$
 - Actualizar: $\beta := \beta - \alpha \nabla J$
 - α es la “tasa de aprendizaje” (qué tan grande es cada paso)

5.4.4 Tasa de Aprendizaje



5.4.5 Formulación Matemática

Para regresión lineal con Error Cuadrático Medio:

$$J(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

El gradiente es: $\nabla J = -\frac{1}{n} X^T (y - \hat{y})$

Y la actualización: $\beta := \beta + \alpha X^T (y - \hat{y})$

5.4.6 Implementación Conceptual

```
import numpy as np

# Inicializar
beta = np.random.randn(n_features)
alpha = 0.01 # tasa de aprendizaje
n_iterations = 1000

# Agregar columna de 1s
X = np.column_stack([np.ones(len(X)), X])

for i in range(n_iterations):
    # Predicción
    y_pred = X @ beta

    # Calcular gradiente
    gradient = -X.T @ (y - y_pred) / len(y)
```

(continues on next page)

(continued from previous page)

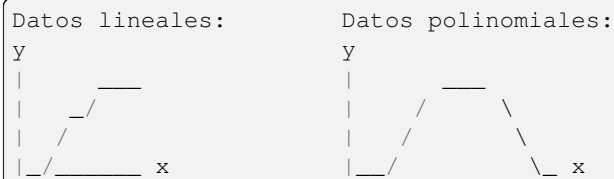
```
# Actualizar pesos
beta = beta - alpha * gradient

# (Opcional) Imprimir pérdida
if i % 100 == 0:
    loss = np.mean((y - y_pred) ** 2)
    print(f"Iteración {i}, Pérdida: {loss}")
```

5.5 Regresión Polinomial

5.5.1 El Problema: Relaciones No Lineales

A veces, la relación entre X e y no es lineal:



Regresión lineal no captura bien esta relación.

5.5.2 La Solución

Crear nuevas características usando potencias de x :

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots$$

Esto se sigue llamando “regresión lineal” porque es lineal en los coeficientes β , aunque sea no lineal en x .

5.5.3 Ejemplo

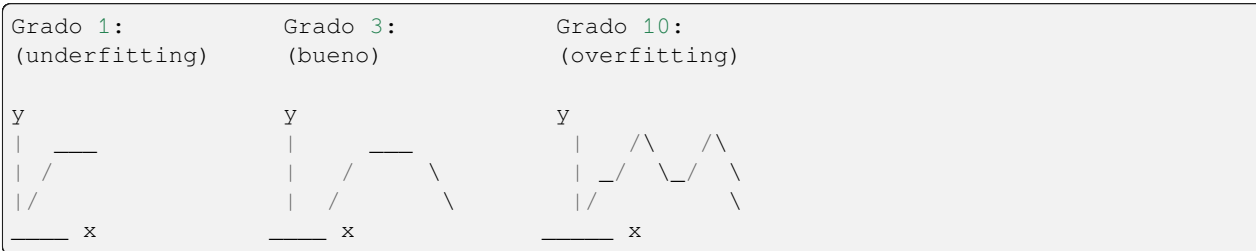
```
# Si X es unidimensional
X_original = X

# Crear características polinomiales
X_poly = np.column_stack([
    X_original,
    X_original ** 2,
    X_original ** 3
])

# Ahora usar regresión lineal en X_poly
# es lo mismo que regresión polinomial
```

5.5.4 Peligro: Overfitting

Polinomios de grado muy alto pueden provocar overfitting:



5.6 Regularización: Ridge y Lasso

5.6.1 El Problema

Los polinomios de alto grado tienen coeficientes muy grandes que capturan el ruido. Queremos penalizar esto.

5.6.2 Ridge Regression (L2)

Agregamos un término de penalización al MSE:

$$J(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \frac{\lambda}{2} \sum_{j=1}^p \beta_j^2$$

Esto **encoge** los coeficientes hacia cero, pero no los hace exactamente cero.

5.6.3 Lasso Regression (L1)

Usa una penalización diferente:

$$J(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

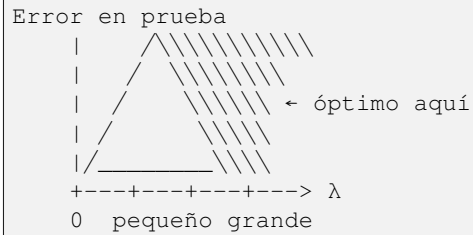
Lasso puede hacer que algunos coeficientes sean **exactamente cero**, lo que es útil para **selección de características**.

5.6.4 Comparación

Aspecto	Ridge	Lasso
Penalización	β^2	$ \beta $
Encoge hacia cero	Suave	Abrupto
Selección de features	No	Sí
Para interpretabilidad	Peor	Mejor

5.6.5 El Parámetro λ

$\lambda = 0$: $\lambda = \text{medio}$: $\lambda = \text{grande}$:
 Sobreajuste Balance Subajuste
 (rojo = error)



λ se elige típicamente usando validación cruzada.

5.7 Resumen

- **Regresión Lineal:** modelo simple pero poderoso
- **OLS:** solución exacta, pero costosa computacionalmente
- **Descenso de Gradiente:** método general, funciona para cualquier modelo
- **Regresión Polinomial:** captura relaciones no lineales
- **Regularización:** previene overfitting (Ridge para retención, Lasso para selección)

5.8 Próximo

En el siguiente capítulo, aprenderemos a extender estas ideas a **Clasificación**, donde predecimos categorías en lugar de números continuos.

2.2 CLASIFICACIÓN

6.1 Introducción

En clasificación, nuestro objetivo es predecir **categorías** (clases) en lugar de valores continuos. Ejemplos incluyen:

- Spam vs No-spam
- Enfermo vs Sano
- Gato vs Perro
- Positivo vs Negativo

6.2 Regresión Logística

6.2.1 De Regresión a Clasificación

Regresión lineal predice valores continuos sin restricción. Para clasificación, necesitamos:

- Salidas entre 0 y 1 (probabilidades)
- Decisiones claras (0 o 1)

Usamos la **función sigmoide** (o logística):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Función Sigmoide:

Probabilidad



Propiedades clave:

- Para $z \rightarrow -\infty$: $\sigma(z) \rightarrow 0$

- Para $z \rightarrow +\infty$: $\sigma(z) \rightarrow 1$
- Para $z = 0$: $\sigma(z) = 0.5$

6.2.2 La Ecuación

Combinamos regresión lineal con la sigmoide:

$$\hat{p} = \sigma(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)$$

Donde $\hat{p}$ es la probabilidad predicha de la clase positiva (1).

6.2.3 Función de Pérdida: Log Loss (Cross-Entropy)

No podemos usar MSE para clasificación (no es apropiado). En su lugar, usamos:

$$J(\beta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

Donde:

- Si $y_i = 1$ (clase positiva real): penalizamos si $\hat{p}_i$ es bajo
- Si $y_i = 0$ (clase negativa real): penalizamos si $\hat{p}_i$ es alto

6.2.4 Decisión

Una vez que tenemos probabilidades, decidimos:

- Si $\hat{p} > 0.5$: predecir clase 1
- Si $\hat{p} \leq 0.5$: predecir clase 0

6.3 Matriz de Confusión

6.3.1 Visualizar el Desempeño

		Predicción	
		Positivo	Negativo
Actual	Positivo	TP	FN
	Negativo	FP	TN

Donde:

- **TP (Verdadero Positivo)**: Correctamente predicho como 1
- **TN (Verdadero Negativo)**: Correctamente predicho como 0
- **FP (Falso Positivo)**: Incorrectamente predicho como 1 (falsa alarma)
- **FN (Falso Negativo)**: Incorrectamente predicho como 0 (falta)

6.3.2 Ejemplo Práctico

Detectar enfermedades:

	Test		
	Positivo	Negativo	
Realidad Positivo	85	15	(85 personas enfermas correctamente identificadas, 15 no detectadas)
Negativo	10	890	(10 falsos positivos, 890 verdaderos negativos)

6.4 Métricas de Evaluación

6.4.1 Precisión (Precision)

$$\text{Precisión} = \frac{TP}{TP + FP}$$

De todos los casos que predijimos positivos, ¿cuántos realmente lo eran?

Uso: Importante cuando los falsos positivos son costosos (ej: spam filtering - no queremos bloquear emails legítimos)

6.4.2 Recall (Sensibilidad)

$$\text{Recall} = \frac{TP}{TP + FN}$$

De todos los casos positivos reales, ¿cuántos encontramos?

Uso: Importante cuando los falsos negativos son costosos (ej: detección de enfermedad - queremos detectar a todos los enfermos)

6.4.3 Precisión vs Recall: Trade-off

Aumentar un generalmente disminuye el otro:

Umbral bajo (predecimos 1 más a menudo):

- Recall: alto (encontramos muchos positivos)
- Precisión: baja (muchos son incorrectos)

Umbral alto (predecimos 1 pocas veces):

- Recall: bajo (nos perdemos muchos positivos)
- Precisión: alta (cuando decimos 1, es correcto)

6.4.4 F1-Score

Combina ambas en una sola métrica:

$$F1 = 2 \times \frac{\text{Precisión} \times \text{Recall}}{\text{Precisión} + \text{Recall}}$$

Es la **media armónica** de precisión y recall.

6.4.5 AUC-ROC

Curva ROC (Receiver Operating Characteristic):

- Grafica Recall (eje y) vs Tasa de Falsos Positivos (eje x)
- A diferentes umbrales de decisión

AUC (Area Under Curve):

- Área bajo la curva ROC
- Rango: 0 a 1
- 1.0 = perfecto
- 0.5 = aleatorio
- < 0.5 = peor que aleatorio

6.5 Árboles de Decisión

6.5.1 Intuición

Un árbol de decisión toma decisiones recursivamente basado en características:

```
¿Horas estudiadas > 3?
|
Si | No
| | |
| | | Calificación = baja
| | |
| | | ¿Dificultad del examen?
| | | |
| | | | Fácil | Difícil
| | | | | |
| | | | | | Calificación = media
| | | | | | Calificación = alta
```

6.5.2 Ventajas

- **Interpretable:** fácil de explicar
- **No paramétrico:** no asume forma de datos
- **Maneja no linealidad:** naturalmente
- **Maneja características mixtas:** numéricas y categóricas

6.5.3 Desventajas

- **Propenso a overfitting:** puede crecer demasiado profundo
- **Inestable:** pequeños cambios en datos pueden cambiar completamente el árbol

6.6 Random Forest

6.6.1 Idea: Ensemble de Árboles

En lugar de un árbol, creamos muchos árboles (por ejemplo, 100) con variación:

- Cada árbol se entrena en una muestra bootstrap (muestreo con reemplazo)
- Cada división usa un subconjunto aleatorio de características

6.6.2 Predicción



Para clasificación: clase que más árboles predicen Para regresión: promedio de predicciones

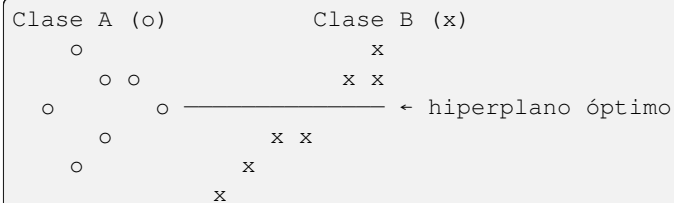
6.6.3 Por Qué Funciona

- **Reduce varianza:** promediar muchos árboles ruidosos
- **Mantiene bajo sesgo:** cada árbol puede ser complejo
- **Captura interacciones:** los árboles las aprenden naturalmente

6.7 Support Vector Machines (SVM)

6.7.1 Objetivo

Encontrar el **hiperplano** (línea en 2D, plano en 3D, etc.) que mejor separa las clases:

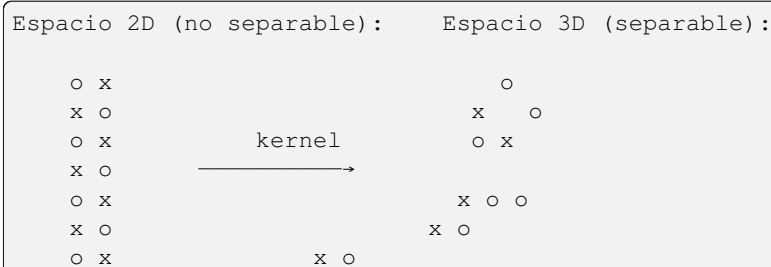


El “margen” es la distancia entre la línea y los puntos más cercanos.

6.7.2 SVM intenta maximizar este margen.

6.7.3 El Kernel Trick

Para datos no linealmente separables, SVM usa kernels para proyectar a dimensiones superiores:



Kernels comunes:

- **Linear:** para datos separables linealmente
- **RBF (Radial Basis Function):** para datos complejos
- **Polynomial:** para patrones polinomiales

6.8 Gradient Boosting (XGBoost, LightGBM)

6.8.1 Idea: Aprender de los Errores

A diferencia de Random Forest que entrena árboles en paralelo, Boosting los entrena **secuencialmente**:

1. Entrenar árbol 1 en datos originales
2. Entrenar árbol 2 enfocándose en errores del árbol 1
3. Entrenar árbol 3 enfocándose en errores combinados
4. ...

```
Árbol 1: predice algo, comete errores
      ↓
Árbol 2: aprende de esos errores
      ↓
Árbol 3: aprende de errores residuales
      ↓
Predicción = suma ponderada de todos
```

6.8.2 Ventajas

- **Muy precisos:** estado del arte en competencias de ML
- **Maneja datos complejos:** captura patrones intrincados
- **Feature importance:** identifica características importantes

6.8.3 Desventajas

- **Lento para entrenar:** secuencial, no paralelo
- **Muchos hiperparámetros:** requiere ajuste cuidadoso
- **Menos interpretable:** más difícil de entender

6.9 Redes Neuronales: Introducción

6.9.1 El Perceptrón

Una neurona artificial que intenta replicar una neurona biológica:

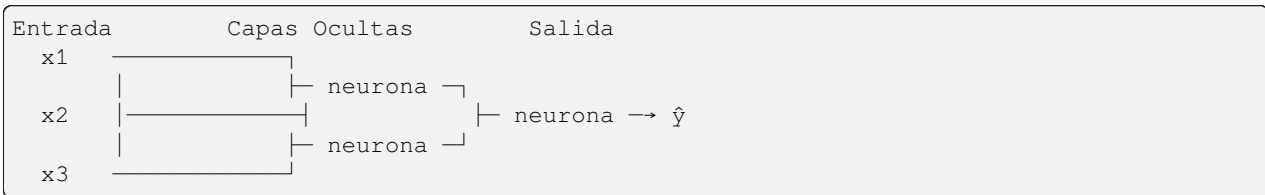
```
Entrada → Sumatorio → Activación → Salida
x1  ───┐
      │ suma ponderada   función
x2  ───┴──→ z = Σ w*x + b → σ(z) → salida
x3  ───┐
```

Donde:

- **w:** pesos (lo que aprende)
- **b:** sesgo (bias)
- **σ:** función de activación (sigmoid, ReLU, etc.)

6.9.2 De una Neurona a Redes

Combinamos múltiples neuronas en capas:



Cada neurona aprende a detectar patrones, capas posteriores combinan estos patrones.

6.9.3 Funciones de Activación

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Rango: [0, 1]
- Antiguo, menos usado ahora

ReLU (Rectified Linear Unit)

$$\text{ReLU}(z) = \max(0, z)$$

- Rápido de computar
- Más usado actualmente

Softmax (para múltiples clases)

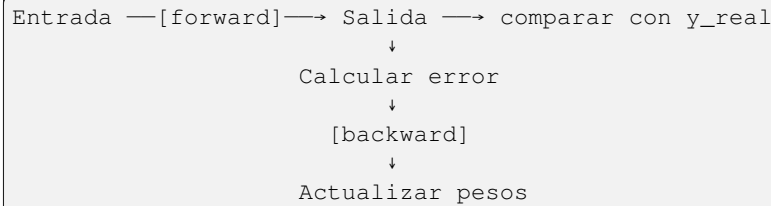
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- Convierte scores en probabilidades
- Las probabilidades suman 1

6.9.4 Entrenamiento: Backpropagation

El algoritmo de entrenamiento de redes neuronales:

1. **Forward Pass:** calcular salida
2. **Calcular pérdida:** qué tan lejos estamos del objetivo
3. **Backward Pass:** propagar el error hacia atrás
4. **Actualizar pesos:** con el error propagado



6.10 Comparación de Algoritmos

Algoritmo	Interpretable	Rápido	Preciso	Overfitting
Logística	✓✓	✓✓	○	○
Árbol	✓✓	✓✓	○	✗✗
SVM	✗	✗	✓✓	✓
Random Forest	○	✓	✓✓	✓
XGBoost	✗	✗	✓✓✓	✓
Red Neuronal	✗✗	✗	✓✓✓	✗

Donde ✓ = bueno, ✗ = malo, ○ = medio

6.11 Pauta para Elegir Algoritmo

1. **Comienza simple:** Regresión Logística
2. **Si necesitas interpretabilidad:** Árbol de Decisión
3. **Si necesitas precisión:** Random Forest o XGBoost
4. **Si tienes datos muy complejos:** Red Neuronal
5. **Siempre:** valida con validación cruzada

6.12 Resumen

- **Regresión Logística:** extensión de regresión lineal para clasificación
- **Matriz de Confusión:** visualiza el desempeño
- **Métricas:** elige según lo que importa (precisión o recall)
- **Árboles de Decisión:** interpretables pero proclives a overfitting
- **Random Forest:** ensemble que reduce overfitting
- **SVM:** encuentra separación óptima con kernel trick
- **Gradient Boosting:** estado del arte, requiere cuidado
- **Redes Neuronales:** poderosas pero complejas

Siguiente: En la Sección 3 exploraremos aprendizaje no supervisado, donde trabajamos con datos sin etiquetas.

Part III

Sección 3 - Aprendizaje No Supervisado

SECCIÓN 3: APRENDIZAJE NO SUPERVISADO

Bienvenido a la tercera y última sección principal del curso. Aquí exploraremos técnicas para extraer patrones de datos **sin etiquetas**.

7.1 ¿Qué es Aprendizaje No Supervisado?

En aprendizaje no supervisado:

- Tenemos **solo características** (X)
- **No tenemos etiquetas** (sin y)
- Queremos **descubrir patrones ocultos**

Es como explorar un territorio desconocido sin un mapa. La máquina debe encontrar su propia estructura.

7.2 Tres Tipos Principales

7.2.1 1. Clustering (Agrupamiento)

Agrupar datos similares:

- Segmentar clientes por comportamiento
- Encontrar comunidades en redes sociales
- Clasificar documentos por tema
- Identificar células cancerosas en imágenes

7.2.2 2. Reducción de Dimensionalidad

Simplificar datos complejos:

- Visualizar datos de alta dimensión
- Eliminar ruido
- Acelerar algoritmos posteriores
- Comprensión interpretable

7.2.3 3. Detección de Anomalías

Identificar casos raros o inusuales:

- Fraude en transacciones
- Fallos en sistemas
- Comportamiento anómalo de usuarios
- Valores atípicos en datos científicos

7.3 ¿Qué Encontrarás en Esta Sección?

7.3.1 Capítulo 3.1: Clustering

- **K-Means**: partición de datos en grupos
- **Método del Codo**: seleccionar número óptimo de clusters
- **Clustering Jerárquico**: dendrogramas y fusiones
- **DBSCAN**: agrupamiento basado en densidad

7.3.2 Capítulo 3.2: Reducción de Dimensionalidad

- **PCA (Principal Component Analysis)**: maximiza varianza explicada
- **t-SNE y UMAP**: visualización no lineal

7.3.3 Capítulo 3.3: Detección de Anomalías

- **Conceptos de patrones inusuales**: qué es una anomalía
- **Métodos de detección**: aplicaciones prácticas

7.4 Desafío Principal: Evaluación

Sin etiquetas, ¿cómo sabemos si nuestros resultados son correctos?

Aprendizaje Supervisado:

Datos → Modelo → Predicción → Comparar con y real → Error

Aprendizaje No Supervisado:

Datos → Modelo → Estructura ??? → ¿Es correcta?

Esto es más desafiante y requiere:

- **Intuición del dominio**: ¿tiene sentido el resultado?
- **Métricas internas**: silhueta, índice Davies-Bouldin
- **Validación externa**: etiquetas disponibles después (si las hay)

7.5 Aplicaciones Reales

7.5.1 E-commerce

Segmentar clientes en grupos para marketing personalizado sin etiquetar manualmente.

7.5.2 Biología

Agrupar proteínas por similitud estructural para descubrir nuevas familias.

7.5.3 Astronomía

Detectar anomalías en observaciones de telescopios (candidatos a eventos raros).

7.5.4 Análisis de Redes

Encontrar comunidades de usuarios con intereses similares en redes sociales.

7.6 Estructura Recomendada

1. **Clustering:** Aprende a agrupar datos
2. **Reducción de Dimensionalidad:** Aprende a simplificar datos complejos
3. **Detección de Anomalías:** Aprende a encontrar patrones raros

7.7 Objetivos de Aprendizaje

Después de completar esta sección, podrás:

- ✓ Implementar K-Means desde cero
- ✓ Elegir el número óptimo de clusters
- ✓ Visualizar datos de alta dimensión
- ✓ Detectar anomalías en datos reales
- ✓ Entender las limitaciones de cada técnica

7.8 Nota Importante

El aprendizaje no supervisado es más **arte que ciencia**. No hay una única respuesta “correcta”. Tu interpretación domain and feedback externo son cruciales.

Empecemos: Dirígete al Capítulo 3.1 para aprender sobre Clustering, el enfoque no supervisado más popular.

3.1 CLUSTERING

8.1 Introducción

Clustering (agrupamiento) es la tarea de dividir datos en grupos (clusters) de manera que:

- Los puntos **dentro de un grupo sean similares** entre sí
- Los puntos en **grupos diferentes sean disimilares**

Es como encontrar familias naturales en tus datos sin que nadie te haya dicho cuáles son.

8.2 K-Means

8.2.1 El Algoritmo Más Simple

K-Means es el algoritmo de clustering más popular por su simplicidad y efectividad.

Idea intuitiva:

1. Escoger K centroides (centros de clusters) aleatorios
2. Asignar cada punto al centroide más cercano
3. Recalcular centroides como el promedio de puntos en cada cluster
4. Repetir hasta convergencia

8.2.2 Visualización

```
Iteración 1:      Iteración 2:      Convergencia:
X X              X X X              X X X
X X      X ♦ X → X X ♦ X          X X ♦ X
      X X      ♦   X X              ♦   X X
      ♦ ♦      X ♦ X X              X ♦ X X
      X  X      X      X ♦          X      X ♦
X ♦  X      X X  X ♦          X X  X ♦

♦ = centroide (centro del cluster)
```

8.2.3 Algoritmo Formal

Entrada: datos X, número de clusters K

1. Inicializar K centroides aleatoriamente
2. Repetir hasta convergencia:
 - a) Asignación: para cada punto, asignar al centroide más cercano
 - b) Actualización: recalcular cada centroide como media de su cluster
3. Retornar: centroides finales y asignaciones

8.2.4 Función Objetivo

K-Means intenta minimizar la **suma de distancias al cuadrado dentro de clusters**:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Donde:

- C_k es el cluster k
- μ_k es el centroide del cluster k
- $\|...\|$ es la distancia euclidiana

8.2.5 Implementación Conceptual

```
import numpy as np

def kmeans(X, K, max_iterations=100):
    # 1. Inicializar centroides aleatoriamente
    centroids = X[np.random.choice(len(X), K, replace=False)]

    for iteration in range(max_iterations):
        # 2. Asignar cada punto al centroide más cercano
        distances = np.zeros((len(X), K))
        for k in range(K):
            distances[:, k] = np.linalg.norm(X - centroids[k], axis=1)
        assignments = np.argmin(distances, axis=1)

        # 3. Guardar centroides antiguos para verificar convergencia
        old_centroids = centroids.copy()

        # 4. Recalcular centroides
        for k in range(K):
            centroids[k] = X[assignments == k].mean(axis=0)

        # 5. Verificar convergencia
        if np.allclose(centroids, old_centroids):
            break

    return centroids, assignments
```

8.2.6 Complejidad

- **Tiempo:** $O(n * K * d * \text{iteraciones})$ donde n = muestras, K = clusters, d = dimensiones
- **Espacio:** $O(n * d)$

Muy eficiente para datos grandes.

8.3 Eligiendo K: El Método del Codo

8.3.1 El Problema

¿Cómo saber cuántos clusters debe haber? K-Means requiere especificar K de antemano.

8.3.2 La Solución: Elbow Method

Ejecutar K-Means para diferentes valores de K (1, 2, 3, ..., 10) y graficar la **inercia** (J):



- Hasta el “codo”: inercia decrece rápido (útil agregar K)
- Después del “codo”: decrece lentamente (no agrega valor)
- Elegir K donde está el “codo”

8.3.3 Interpretación

$K=1$: Todos en un grupo (alta inercia)
 $K=3$: Grupos naturales emergen
 $K=10$: Clusters innecesarios (overfitting)

8.3.4 Implementación

```
inertias = []
K_range = range(1, 11)

for K in K_range:
    centroids, assignments = kmeans(X, K)
    inertia = calculate_inertia(X, centroids, assignments)
    inertias.append(inertia)

# Graficar
import matplotlib.pyplot as plt
plt.plot(K_range, inertias, 'bo-')
plt.xlabel('K')
plt.ylabel('Inercia')
plt.show()
```

8.4 Clustering Jerárquico

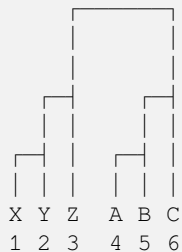
8.4.1 Alternativa: Construir un Árbol

En lugar de decidir K de antemano, construimos una jerarquía de clusters:

```
Todos los datos
|
(fusionar)
|
Cluster 1   Cluster 2
|           |
(fusionar)  (fusionar)
|           |
C1.1 C1.2  C2.1 C2.2 C2.3
```

8.4.2 Dendrograma

Una visualización de este árbol:



Cortando horizontalmente en diferentes alturas → diferentes números de clusters.

8.4.3 Métodos de Enlace (Linkage)

¿Cómo medimos distancia entre clusters?

Single Linkage: distancia entre los puntos más cercanos

```
Cluster A      Cluster B
  X ... X
  ← mínima distancia
```

- Puede crear clusters largos y delgados

Complete Linkage: distancia entre los puntos más lejanos

```
Cluster A      Cluster B
  X           X
  ← máxima distancia

  X           X
```

- Tiende a crear clusters compactos

Average Linkage: promedio de distancias

- Equilibrio entre ambos

Ward Linkage: minimiza incremento de varianza

- Generalmente produce buenos resultados

8.4.4 Ventajas vs K-Means

Aspecto	K-Means	Jerárquico
K previo	Requerido	No
Visualización	Difícil	Dendrograma
Escalabilidad	Excelente	Pobre ($O(n^2)$)
Clusters globales	Mejores	Específicos

8.5 DBSCAN (Density-Based Spatial Clustering)

8.5.1 Alternativa: Clustering por Densidad

A diferencia de K-Means que asume clusters esféricos, DBSCAN:

- Agrupa puntos en regiones densas
- Marca puntos aislados como outliers
- Descubre clusters de cualquier forma

8.5.2 Conceptos Clave

Vecindad (ϵ -neighborhood): todos los puntos dentro de distancia ϵ

Punto X:	Vecindad ϵ :
o	◆ = punto core (muchos vecinos)
o X o	○ = punto frontera (pocos vecinos)
o o	◇ = outlier (muy aislado)

Punto Core: tiene $\geq \text{min_samples}$ puntos en su ϵ -vecindad

Punto Frontera: no es core pero está en la vecindad de un core

Outlier: ni core ni frontera

8.5.3 El Algoritmo

```
Para cada punto no visitado:
  Si es punto core:
    Crear nuevo cluster
    Agregar todos sus vecinos recursivamente
  Si no es core:
    Marcar como frontera u outlier
```

8.5.4 Ventajas

- **Clusters arbitrarios:** no asume forma
- **Detecta outliers:** automáticamente
- **No requiere K:** parámetros ϵ y min_samples en su lugar
- **Escalable:** con índices espaciales

8.5.5 Desventajas

- **Sensible a parámetros:** elegir ϵ y min_samples es difícil
- **Datos de alta dimensión:** distancias menos significativas
- **Densidades variadas:** difícil si clusters tienen densidades diferentes

8.5.6 Comparación de Algoritmos

Datos Circulares:	Datos Complejos:	Datos con Outliers:
K-Means:	K-Means:	K-Means:
O O O	X X X	X X X
O O O ✓	X X X (malo)	X X X (outlier etiquetado)
O O O	X X	
DBSCAN:	DBSCAN:	DBSCAN:
O O O	X X X	X X X

(continues on next page)

(continued from previous page)

○ ○ ○	✓		X	X	X	✓		X	X	X	✓	(outlier detectado)
○ ○ ○			X		X							

8.6 Métrica de Evaluación: Silhueta

Sin etiquetas verdaderas, ¿qué tan bueno es nuestro clustering?

Coficiente de Silhueta:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

Donde:

- a_i = distancia promedio a otros puntos en su cluster
- b_i = distancia promedio al cluster más cercano

Interpretación:

- $s = 1$: punto está bien en su cluster
- $s = 0$: punto está en el borde
- $s = -1$: punto está en el cluster equivocado

Rango: $[-1, 1]$, más alto es mejor.

8.7 Resumen

- **K-Means**: simple, rápido, requiere K conocido
- **Método del Codo**: heurística para elegir K
- **Clustering Jerárquico**: dendrograma, mejor visualización
- **DBSCAN**: detecta formas arbitrarias y outliers
- **Silhueta**: métrica para evaluar calidad sin etiquetas

Siguiente: En 3.2 aprenderemos a reducir dimensionalidad para simplificar y visualizar datos complejos.

3.2 REDUCCIÓN DE DIMENSIONALIDAD

9.1 Introducción

Muchos datasets tienen cientos o miles de características. Trabajar con tantas dimensiones es:

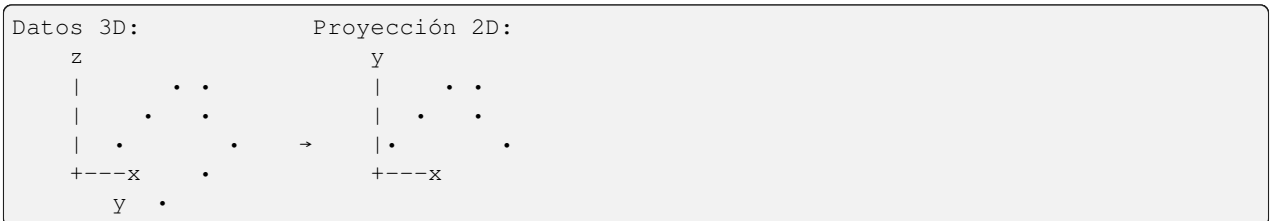
- **Computacionalmente costoso:** algoritmos más lentos
- **Difícil de visualizar:** no podemos ver datos de 100 dimensiones
- **Ruidoso:** muchas características irrelevantes
- **Maldición de la dimensionalidad:** modelos generalizan peor

La reducción de dimensionalidad **comprime datos complejos** en pocas dimensiones manteniendo información esencial.

9.2 PCA: Principal Component Analysis

9.2.1 Intuición

Imagina que tienes datos en 3D. Si la mayoría de la variación ocurre en 2 direcciones principales, puedes proyectar a esas 2 dimensiones sin perder mucha información.



9.2.2 Componentes Principales

Los “componentes principales” son direcciones donde los datos varían más.

- **PC1:** dirección de máxima varianza
- **PC2:** dirección de segunda mayor varianza, ortogonal a PC1
- **PC3:** tercera dirección, ortogonal a PC1 y PC2
- ...

Varianza en diferentes direcciones:

Alta varianza PC1 (máxima)
 →
 /
 / PC2 (ortogonal)
 →

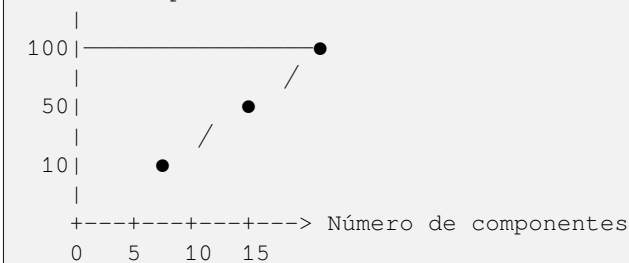
9.2.3 Algoritmo Formal

1. **Centrar datos:** restar la media
2. **Calcular matriz de covarianza:** qué características varían juntas
3. **Descomposición en valores propios:** encontrar direcciones principales
4. **Proyectar:** multiplicar datos por los vectores propios

9.2.4 Varianza Explicada

Cada componente explica una cierta porcentaje de la varianza total:

Varianza explicada acumulada:



Decisión: ¿cuántos componentes mantener?

- Mantener 95% de varianza explicada es típico
- Reduce dimensionalidad pero conserva estructura

9.2.5 Implementación Conceptual

```
import numpy as np

def pca(X, n_components):
    # 1. Centrar datos
    X_centered = X - X.mean(axis=0)

    # 2. Matriz de covarianza
    cov_matrix = np.cov(X_centered.T)

    # 3. Descomposición en valores propios
    eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

    # 4. Ordenar por valor propio (varianza explicada)
```

(continues on next page)

(continued from previous page)

```
idx = eigenvalues.argsort()[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

# 5. Seleccionar top n_components
components = eigenvectors[:, :n_components]

# 6. Proyectar datos
X_reduced = X_centered @ components

return X_reduced, components, eigenvalues
```

9.2.6 Interpretación

Aunque PCA reduce dimensiones, las nuevas características (componentes) son **combinaciones de las originales**:

```
PC1 = 0.7 * Feature1 + 0.3 * Feature2 - 0.1 * Feature3
PC2 = 0.2 * Feature1 - 0.8 * Feature2 + 0.5 * Feature3
```

Es difícil interpretar qué significa cada componente.

9.2.7 Ventajas y Desventajas

Aspecto	PCA
Rápido	✓✓
Interpretable	✗ (componentes son mezclas)
Lineal	Solo captura relaciones lineales
Óptimo	Maximiza varianza (no siempre lo correcto)

9.3 t-SNE (t-Distributed Stochastic Neighbor Embedding)

9.3.1 Problema con PCA

PCA es lineal. Para datos complejos con estructura no lineal, falla:

Datos en espiral PCA (fallida): t-SNE (buena):

9.3.2 Idea: Conservar Similaridades Locales

En lugar de conservar varianza global, t-SNE intenta:

- Puntos similares en espacio alto → cercanos en espacio bajo
- Puntos disímiles en espacio alto → lejanos en espacio bajo

9.3.3 Propiedades

- **No lineal:** puede descubrir estructuras complejas
- **Preserva estructura local:** lo que ves es real localmente
- **Pero:** distancias globales no son significativas

```
❌ NO ES CORRECTO decir:  
"Los dos clusters están a distancia X"  
"El cluster A es más grande que B"  
  
✓ ES CORRECTO decir:  
"Los puntos A y B están en el mismo vecindario"  
"Hay dos grupos distintos"
```

9.3.4 Parámetros Clave

- **perplexity:** número de vecinos a considerar (típicamente 30-50)
- **learning_rate:** velocidad de optimización
- **n_iterations:** cuántas iteraciones ejecutar

Estos parámetros afectan el resultado final significativamente.

9.3.5 Interpretación

```
t-SNE 2D de dataset MNIST (dígitos 0-9):
```

```
0 0 0 0  
0 0 1 1 1  
0 1 1 1 1 1  
2 2 2 1  
2 2 2 3 3 3 3  
2 2 3 3 3 3  
4 4 4 4 3  
4 4 4 4 4  
5 5 5 5  
5 5 5 5  
6 6 6 6 6 6  
6 6 6 6  
7 7 7  
7 7 7 7 7  
8 8 8 8  
8 8 8 8 8 8  
9 9 9 9  
9 9 9 9
```

Vemos dígitos similares agrupados naturalmente.

9.4 UMAP (Uniform Manifold Approximation and Projection)

9.4.1 Mejora sobre t-SNE

UMAP es similar a t-SNE pero con ventajas:

- **Más rápido:** especialmente para datasets grandes
- **Preserva escala global mejor:** distancias globales más significativas
- **Más estable:** resultados reproducibles con mismos parámetros

9.4.2 Cuándo Usar Qué

Situación	Recomendar
Visualizar datos complejos	t-SNE o UMAP
Dataset muy grande (>10k)	UMAP (más rápido)
Necesitas interpretación global	PCA
Necesitas velocidad	PCA
Dataset pequeño (<1k)	t-SNE

9.5 Aplicaciones de Reducción de Dimensionalidad

9.5.1 1. Compresión de Datos

```
Imágenes 28×28 = 784 dimensiones
PCA a 50 componentes = 93% varianza explicada
Compresión 15x con mínima pérdida visual
```

9.5.2 2. Visualización para Exploración

```
Datos de 100 dimensiones
Aplicar t-SNE a 2D
Visualizar y detectar clusters naturales
```

9.5.3 3. Preprocesamiento

```
Datos ruidosos 1000D
Aplicar PCA mantener 95% varianza
Reduce ruido y acelera algoritmos posteriores
```

9.5.4 4. Feature Engineering

```
Crear nuevas características usando componentes
Como inputs a modelos posteriores
```

9.6 Maldición de la Dimensionalidad

Entender por qué reducción es importante:

```
En 1D (línea):
Puntos aleatorios: X - X - - X - X -
Unos están cercanos, otros lejanos

En 2D (plano):
Puntos aleatorios: X - - - X - X - -
                   - - - - - - - -
                   X - - - - X - - -
Ahora más separados

En 1000D (hipercubo):
Puntos aleatorios: CASI TODOS estabAn equidistantes
Distancia entre puntos aleatorios ≈ constante
No hay información de proximidad
```

Conclusión: en alta dimensión, todos los puntos se vuelven casi equidistantes. La noción de “cercano” desaparece.

9.7 Resumen

- **PCA:** rápido, lineal, interpatable características originales
- **t-SNE:** visualización excelente, no lineal, lento para datos grandes
- **UMAP:** balance entre PCA y t-SNE
- **Varianza explicada:** métrica para elegir número de componentes
- **Maldición de la dimensionalidad:** motiva reducción
- **Aplicaciones:** compresión, visualización, preprocesamiento

Siguiente: En 3.3 aprenderemos a detectar anomalías, el último tema en aprendizaje no supervisado.

3.3 DETECCIÓN DE ANOMALÍAS

10.1 Introducción

Las anomalías son puntos de datos que se desvían significativamente del comportamiento “normal”.

Datos Normales:		Con Anomalías:
X X		X X
X X	→	X X
X X		X X
		◆ (anomalía)

La detección de anomalías tiene aplicaciones críticas:

- **Seguridad:** fraude en transacciones bancarias
- **Mantenimiento:** fallos incipientes en maquinaria
- **Biología:** mutaciones genéticas raras
- **Astronomía:** eventos cósmicos inusuales

10.2 Conceptos Fundamentales

10.2.1 ¿Qué es una Anomalía?

Una observación que:

1. Es rara (ocurre infrecuentemente)
2. Es diferente del patrón normal
3. Sugiere un proceso generativo diferente

Distribución Normal + Anomalías:

Densidad

◆ anomalía 1

◆ anomalía 2

+---+---+---+---> Valor

10.2.2 Tipos de Anomalías

Punto (Point Anomaly)

Un punto individual anómalo en el contexto de todos los datos.

```
[normal, normal, anomalía, normal, normal]
```

Contextual (Contextual Anomaly)

Normal en general, pero anómalo en este contexto específico.

```
Temperatura en Salamanca:
Enero: 2°C (normal)
Julio: 35°C (normal)
Diciembre: 25°C (!anomalía!)
```

Colectivo (Collective Anomaly)

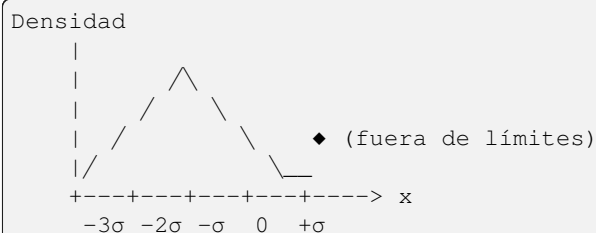
Una colección de puntos es anómala, aunque individualmente sean normales.

```
Patrón de fraude: transacción normal + transferencia normal
Juntas forman patrón de lavado de dinero
```

10.3 Métodos de Detección

10.3.1 1. Estadístico: Gaussiana Univariada

Asumir que datos normales siguen distribución normal:



Algoritmo:

1. Estimar media μ y desviación σ de datos normales
2. Para nuevo punto x , calcular probabilidad $P(x)$
3. Si $P(x) < \text{umbral}$, es anomalía

$$P(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Regla práctica: puntos a $\pm 3\sigma$ de la media son anomalías (99.7% de datos).

Limitaciones: solo funciona bien para una dimensión. En alta dimensión, la mayoría de puntos están “en la cola”.

10.3.2 2. Basado en Distancia: Aislamiento por Proximidad

Si un punto está muy alejado de sus vecinos → anomalía.

Punto normal:	Anomalía:
<pre> o o o o → o o o o o ♦ (cerca) </pre>	<pre> o o o o ♦ (aislado) </pre>

Métodos:

- **K-Nearest Neighbors (k-NN)**: punto con vecinos lejanos
- **LOF (Local Outlier Factor)**: compara densidad local con vecinos
- **Isolation Forest**: árbol que aísla puntos anómalos

10.3.3 3. Aislamiento (Isolation Forest)

Idea: los puntos anómalos son más fáciles de aislar.

```

Partición 1: Dividir por feature_1
[Normal] | [Normal, Anomalía]

Partición 2: Dividir por feature_2 (en rama con anomalía)
[Normal] | [Anomalía]

Anomalía aislada en 2 particiones
Normal requeriría 10+ particiones

```

Ventajas:

- Eficiente: $O(n \log n)$
- No paramétrico
- Funciona bien en alta dimensión
- Naturalmente detecta puntos anómalos

Desventajas:

- Menos interpretable
- Sensible a randomización

10.3.4 4. Basado en Densidad: Local Outlier Factor (LOF)

Compara densidad local con densidad de vecinos:

Región densa:	Región rara:
<pre> X X X X X X densidad X X X X alta </pre>	<pre> X X X X X (baja densidad) X X X </pre>

LOF = 1: densidad similar a vecinos (normal) LOF > 1: menos denso que vecinos (anomalía)

$$\text{LOF}(p) = \frac{\text{densidad media de vecinos}}{\text{densidad de } p}$$

10.3.5 5. Autoencoders

Red neuronal que:

1. Comprime datos en representación de bajo rango
2. Reconstruye desde esa representación

```
Entrada → Compresión → Reconstrucción → Salida
x          z (100D)      x'
          (10D)

Error de reconstrucción = ||x - x'||
Si error > umbral → anomalía
```

Intuición:

- Datos normales: reconstrucción buena
- Datos anómalos: reconstrucción pobre (nunca los vio)

Ventaja: aprende qué es “normal” de los datos

10.4 Evaluación de Anomalías

10.4.1 El Desafío

Típicamente hay muy pocas anomalías verdaderas:

```
1000 transacciones: 999 normales, 1 fraude
Exactitud de decir "todo es normal": 99.9%
Pero eso es inútil, no detectó el fraude
```

10.4.2 Métricas Adecuadas

Recall (Sensibilidad): $\text{Recall} = \frac{\text{Anomalías detectadas}}{\text{Todas las anomalías}}$

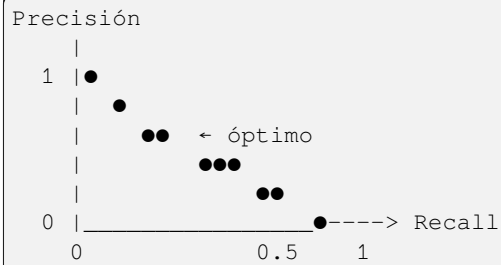
Crítico: queremos encontrar todas las anomalías.

Precisión: $\text{Precisión} = \frac{\text{Anomalías correctas}}{\text{Predichas como anomalía}}$

Si es muy bajo, investigar muchos falsos positivos.

F1-Score: balance entre ambas.

10.4.3 Curva Precision-Recall



A diferencia de ROC, esta es más informativa para clases desbalanceadas.

10.5 Aplicaciones Reales

10.5.1 Detección de Fraude Bancario

Datos normales:

- Transacciones pequeñas (< 500€)
- Horarios de trabajo
- Ubicación consistente

Anomalías:

- Transacción grande (50000€) a las 2 AM
- Ubicación cambió 1000km en 2 horas
- Múltiples transacciones rechazadas luego aceptada

10.5.2 Mantenimiento Predictivo

Sensor de vibración en máquina:

- Normal: 10-20 Hz
 - Anomalía: picos > 50 Hz
- Indicador de fallo incipiente

Acción: mantenimiento preventivo antes de fallo

Ahorro: evita paradas inesperadas

10.5.3 Detección de Intrusiones

Flujo de red normal:

- Pocos intentos de conexión por segundo
- Puertos estándares (80, 443)
- Volumen consistente

Anomalía:

- Escaneo de puertos (múltiples puertos en segundos)
- Volumen explosivo de tráfico
- Puertos inusuales

Acción: bloquear conexión, alertar

10.6 Implementación Conceptual: Isolation Forest

```
import numpy as np

class IsolationForest:
    def __init__(self, n_trees=100, max_depth=20):
        self.n_trees = n_trees
        self.max_depth = max_depth
        self.trees = []

    def fit(self, X):
        for _ in range(self.n_trees):
            tree = self._build_tree(X, depth=0)
            self.trees.append(tree)

    def _build_tree(self, X, depth):
        if depth >= self.max_depth or len(X) <= 1:
            return {'type': 'leaf', 'size': len(X)}

        # Elegir feature aleatorio
        feature = np.random.randint(0, X.shape[1])

        # Elegir valor de split aleatorio
        split_value = np.random.uniform(X[:, feature].min(),
                                         X[:, feature].max())

        # Dividir datos
        left_idx = X[:, feature] < split_value
        right_idx = ~left_idx

        return {
            'type': 'internal',
            'feature': feature,
            'split': split_value,
            'left': self._build_tree(X[left_idx], depth + 1),
            'right': self._build_tree(X[right_idx], depth + 1)
        }

    def predict(self, X):
        anomaly_scores = []
        for x in X:
            # Calcular profundidad promedio en árboles
            depths = [self._traverse(x, tree, 0)
                      for tree in self.trees]
            avg_depth = np.mean(depths)
            anomaly_scores.append(avg_depth)

        # Normalizar: bajo score = anomalía
        return np.array(anomaly_scores)

    def _traverse(self, x, node, depth):
        if node['type'] == 'leaf':
            return depth

        if x[node['feature']] < node['split']:
            return self._traverse(x, node['left'], depth + 1)
        else:
```

(continues on next page)

(continued from previous page)

```
return self._traverse(x, node['right'], depth + 1)
```

10.7 Comparación de Métodos

Método	Complejidad	Lineal	Interpretable	Alto-D
Gaussiana	$O(n)$	✓	✓✓	✗
k-NN	$O(n^2)$	✗	○	✗
LOF	$O(n^2)$	✗	○	✗
Isolation Forest	$O(n \log n)$	✗	○	✓
Autoencoder	$O(n)$	✗	✗	✓✓

10.8 Resumen

- **Anomalías:** eventos raros, contextuales o colectivos
- **Métodos simples:** gaussiana para 1D, k-NN para distancia
- **Métodos avanzados:** Isolation Forest, Autoencoders
- **Evaluación:** Recall más importante que Exactitud
- **Aplicaciones:** fraude, mantenimiento, seguridad

10.9 Reflexión Final

La detección de anomalías es tanto **arte como ciencia**:

- La ciencia: algoritmos probados
- El arte: entender tu dominio para definir qué es “anómalo”

Una anomalía estadística puede ser perfecto en tu negocio. Una anomalía empresarial puede ser perfectamente normal estadísticamente.

¡Felicidades! Has completado el curso de Fundamentos e Introducción al Aprendizaje Automático. Ahora tienes bases sólidas para explorar temas más avanzados como **aprendizaje profundo**, **procesamiento de lenguaje natural**, o **visión por computadora**.

Próximos pasos recomendados:

1. Practicar con datasets reales (Kaggle, UCI)
2. Implementar algoritmos desde cero
3. Leer papers académicos
4. Contribuir a proyectos open-source